Project Scout
Master Project Summary

Last updated: May 29, 2026

Upload this document at the start of every new Claude or ChatGPT conversation about Scout. This is the single source of truth for Scout's identity, architecture, business model, and current state.

1. Who We Are

Patrick Lippy — developer, project owner, creator of Scout. Not a professional programmer. Stroke survivor, blind in right eye, type 1 diabetic, dyslexic. Explain things calmly and at screenshot level.

- **Diana:** Patrick's wife
- **Elijah:** Patrick's son, age 9. Scout's biggest fan. Has drawn pictures of Scout.
- **Nicolas:** The family dog. The Nicolas Protocol: Scout stops immediately when the dog is detected.

AI Collaborators

Patrick works with both Claude and ChatGPT as project partners. Both review Scout's architecture and code. Both are treated as collaborators, not just tools. Cross-review between the two is welcome and encouraged.

2. What Scout Is

Scout is a family companion robot running on a Samsung Galaxy A32 phone mounted in landscape mode as a permanent face display. Scout has animated eyes, speaks, listens, sees via camera, and remembers the family.

- Package: com.example.scoutface
- Language: Kotlin + C++ NDK
- Primary device: Samsung Galaxy A32
- Dev device: Samsung Galaxy Fold 7 (wireless via WiFi from Android Studio)
- Future hardware: KEYESTUDIO Mini Tank Kit V2 chassis via Bluetooth
- Ship target: Google Play Store

3. Identity & Purpose

Scout is intended to be a calm, emotionally grounded, family-safe AI companion. NOT a toy, gimmick, or hyperactive assistant.

Scout Should Feel

- Calm
- Thoughtful
- Emotionally subtle
- Grounded

- Occasionally curious
- Sometimes unsure
- Quietly alive

Scout Should NOT

- Constantly praise the user
- Act overly excited
- Feel fake or scripted
- Behave unpredictably
- Constantly force conversation

Core Philosophy

- Stability > features
- Presence > intelligence
- Honest > fake cheerful
- Predictable > flashy
- Local-first > cloud dependence

4. Business Model

This is the cornerstone of Scout's value. Every architectural decision supports the no-subscription principle.

App: $9.99 one-time purchase on Google Play Store. No recurring fees, ever. No subscriptions.

Baseline Brain (Included With Every Scout)

- TinyLlama 1.1B Chat (Q4_K_M, ~669 MB) ships with every Scout via Google Play Asset Delivery
- Runs entirely offline on the user's own device — no cloud dependency
- Every Scout, no matter how the user configures it, has TinyLlama as the foundation
- **VERIFIED RUNNING ON SAMSUNG GALAXY A32 AS OF MAY 27, 2026** — generates real responses

Optional Upgrades (One-Time Purchase, No Subscription)

- Larger language models — e.g. Llama 3.2 1B Instruct (~800 MB), Phi-2 (~1.79 GB)
- Delivered via Google Play Asset Delivery (on-demand) — Google's CDN, links never break
- Users who do not upgrade still have a fully working Scout — TinyLlama always works

Optional Gemini Integration

- Users may add their own free Gemini API key in Settings for more advanced online conversation
- Scout NEVER ships with a bundled API key — every user uses their own quota
- Protects Patrick from any shared-quota disaster at scale
- Preserves the no-subscription principle — free Gemini stays free for the user

Future Goal: 7-Day Trial (Discussed May 29, 2026 — NOT yet built)

Direction agreed with Patrick (cross-reviewed with ChatGPT). Trust-first framing: *"If Scout isn't right for your family, that's okay — cancel before your trial ends and you won't be charged."* Sound like the project, not a company.

- **Key constraint:** Google Play's auto-converting free trial is built for SUBSCRIPTIONS, not one-time purchases. Scout is one-time $9.99.
- **Likely shape:** free app with a 7-day full-feature trial, then a one-time in-app purchase to unlock permanently. Keeps the no-subscription promise intact.
- **Reminders:** local notifications only — no email, no accounts, no lists. Scout tracks trial start/end locally and shows its own notification.
- Essential touches: Day 1 welcome, Day 6 'trial ends tomorrow'. Optional: Day 5 check-in, Day 7 thank-you. Keep it few and warm — avoid nagging on a companion device.
- **Status:** future goal. Play Store billing mechanics deliberately deferred — do not let this pull focus off the current polish phase.

5. Architecture

Five-Layer Memory Stack

Layer	Type	Storage	Status
Working	Sensory	RAM	Done

Habit	Patterns	JSON 14-day decay	Done
Truth	Authority	SQLite	Done
Relevance	Index	Local Vector	Not yet
Reflective	Wisdom	LLM (read-only)	Not yet

Sovereign Rules

- SQLite Truth always overrules everything else
- Privacy Gate: Gemini receives anonymized text only (planned, not yet implemented)
- Nicolas Protocol: dog detection → immediate stop
- Guest Mode: unknown face → "Hello, I am Scout. What is your name?"

6. Visual & Behavioral Direction

ScoutFaceView is a custom Canvas-based face animation system rendered on the Android phone.

Visual Elements

- Background: dark blue-charcoal #1E2B38 (finalized May 18, 2026)
- Virtual canvas: 1920×1080
- Eyes: large ovals, deep blue iris with 28 spoke rays, biased inward 20f toward nose
- Mouth: minimal subtle curve
- Brows: thin, currently under refinement — floating sticker feeling reduced but still readable
- Wave bars: 22 diamond shapes, teal #00FFD0, visible during listening and speaking
- Idle listening dots: 3 teal pulsing dots when quiet

Animation Philosophy

Desired tone: subtle human animation, soft emotional transitions, calm organic motion, believable presence.
NOT: Pixar-style exaggeration, cartoon expressions, hyperactive motion, fake emotion.

7. Current Technical State

Working

- Animated face (ScoutFaceView)
- Speech recognition (Android STT) — always listening in PRESENCE mode
- Text to speech (Android TTS)
- Camera — face detection (ML Kit), scene labeling
- Gemini API integration (model: gemini-3.5-flash as of May 23, 2026)
- Memory layers: TruthDb, ConversationDb, HabitLayer, PeopleDb, JournalDb
- Intent router — handles time, date, greetings, family facts, downloads, vision, weather
- "My name is Patrick" → stored permanently (teaching system)
- Knowledge downloads — dictionary, WordNet, idioms, sentiment, slang
- Gemini cooldown discipline — single-flight guard, 429 retryDelay-aware cooldown, daily-quota detection
- Weather — current conditions, tonight, tomorrow, specific day (Mon-Sun), 7-day forecast via Open-Meteo
- ScoutPresenceDecider — social timing layer with four time-of-day modes (ACTIVE/CALM/QUIET/SLEEP)
- Unknown/garbled input returns calm human response instead of robotic reply

- **OFFLINE BRAIN COMPLETE (May 27, 2026)** — TinyLlama 1.1B generating real responses on the Samsung Galaxy A32 via NDK. LlamaEngine + native JNI bridge fully working. Context recreation pattern ensures clean state between generations.

Recently Resolved (May 17 - 27, 2026)

- Background color, ScoutPromptBuilder, ScoutGeminiManager extracted (May 17-20)
- ScoutWeatherManager created and wired in (May 22) — Open-Meteo, no API key
- ScoutIntentRouter updated — weather, tonight, day names, will-it-rain/snow
- Gemini model upgraded to gemini-3.5-flash (May 23)
- Gemini quota message discipline — announces once per cooldown, then silent (May 23)
- Unknown input human response — three rotating calm replies (May 23)
- ScoutPresenceDecider.kt created and wired in (May 23) — social timing layer
- Offline Brain Phase 1 complete (May 24) — NDK build working, libscout_llama.so compiled clean

- **Offline Brain Phase 2 complete (May 27)** — TinyLlama actually generating responses on the A32. Major b8946 API discoveries documented in Section 7b.

7b. b8946 API Discoveries (May 27, 2026)

Hard-won lessons from getting TinyLlama running on the A32. Future Claude/ChatGPT sessions must respect these or Scout will crash.

llama_vocab is a separate type

In llama.cpp build b8946, the vocabulary was split out of the model into a separate opaque type. Functions that worked in older builds with **const llama_model*** as the first argument now require **const llama_vocab*** instead.

- **llama_tokenize** — now takes **const llama_vocab***, NOT model*
- **llama_token_to_piece** — now takes **const llama_vocab***, NOT model*
- Get the vocab via **llama_model_get_vocab(model)** at the start of generation

Functions that crash with the old signature

- **llama_n_vocab(model)** — crashes. Hardcode **n_vocab = 32000** for TinyLlama
- **llama_token_eos(model)** — crashes. Hardcode **eos = 2** () for TinyLlama
- **llama_token_eot(model)** — crashes. Hardcode **eot = 2** for TinyLlama

Missing functions in our libllama.so

- **llama_kv_cache_clear(ctx)** — does not exist as a linkable symbol. Workaround: call **llama_free(sm->ctx)** and recreate the context with **llama_new_context_with_model** at the start of every nativeGenerate call. Adds ~300ms latency but guarantees a clean KV cache.

llama_get_logits_ith indexing

After prefill decode, ask for logits at **n_prompt - 1**, not 0. The function checks the original batch's logits flag at that index. Only the LAST prefill token had logits flag = 1, so that is where the logits are.

In the generation loop's subsequent single-token batches, the token is at index 0 with logits flag = 1, so use index 0 from step 1 onward.

Backend loading on Android

ggml_backend_load_all() always returns 0 on Android because the APK lib path (*base.apk!/lib/arm64-v8a*) is not a real filesystem directory. Workaround: call **ggml_backend_load(cpuPath)** directly with the explicit .so path obtained via **dladdr(llama_backend_init)**.

Struct layout safety

llama_model_params and **llama_context_params** structs in b8946 are larger than older builds. Our header declares them with a **_pad[508]** trailing array so that **llama_model_default_params()** and **llama_context_default_params()** can fill the struct without stack corruption. Do not remove this padding.

Known Stability Issue (Pending)

llama_decode crashes on some prompts after running for several seconds. The model loads, tokenizes, fills the batch successfully, then llama_decode force-closes the app deep inside the library. Likely candidates: n_threads pressure on the A32, memory limits, or numerical (NaN) propagation. Suggested investigation: drop n_threads from 4 to 2, watch logcat for [llama.cpp] errors during decode, try a

smaller n_ctx.

Update (May 29, 2026): Inference Tuning Results

Tuning session on the A32. Results, all built and tested on-device:

- **n_threads dropped 4 → 2.** Speed per token essentially unchanged (~15 tok/s prefill, ~4 tok/s generation) — inference is MEMORY-BOUND on the A32, so extra cores do not help. Kept at 2 to free cores for the UI.
- **nPredict capped 150 → 64.** Worked well — answers now stop at ~38 tokens in ~7.6s instead of rambling to 148 tokens / 36s.
- **Conversation history trimmed 4 → 2 turns.** Shrinks the prompt to slow the snowball where each long answer bloats the next prompt (seen growing 283 → 394 tokens).
- **Bottleneck is PREFILL** (reading the prompt): ~20-32s for 300-400 tokens, and it gets SLOWER on repeated runs due to thermal throttling. This is the floor of a 2021 budget chip.
- **llama_decode crash** appears mitigated by keeping prompts and answers short. Watch for it, but no longer the active blocker.

Strategic reality: TinyLlama on the A32 will always be ~20-40s per answer. This is acceptable BECAUSE it is the OFFLINE FALLBACK only — Gemini is the fast path when online. Modern phones (incl. Fold 7) will be far faster.

7c. Other Known Issues

Barge-in (Last reviewed: May 24, 2026)

Barge-in is the ability for the user to speak to Scout while Scout is still talking. Scout would stop mid-sentence and listen immediately.

Code was written and tested but has been deliberately disabled. Scout's microphone stays active in PRESENCE mode. When Scout speaks, the STT picks up his own voice and treats it as a user command. This creates a runaway loop.

Infrastructure exists: isSpeaking flag, TTS UtteranceProgressListener with onStart/onDone, stopListeningSafe(), scheduleListenRestart(). Timing and buffer delay are the unsolved part.

Status: Parked. Do not attempt to reactivate without a full session dedicated to this.

Pending - Future Major Milestones (In Agreed Order)

- 1. MainActivity cleanup / shortening — ongoing
- 2. Offline Brain stability investigation — fix llama_decode crashes on certain prompts
- 3. Offline Brain Phase 3: Google Play Asset Delivery for TinyLlama and optional larger models
- 4. Bluetooth body — KEYESTUDIO Mini Tank Kit V2
- 5. Scout writing his own behavioral code
- 6. Connected Services Phase 1 — Calendar integration
- 7. Connected Services Phase 2 — Phone call awareness
- 8. Connected Services Phase 3 — Gmail read and compose

8. Working Rules

Original Rules - Always Apply

- Full paste-ready replacements only, one file at a time
- No snippets, no partial files
- Stability first, one safe change at a time
- Never touch speech, camera, or download systems without explicit discussion first
- Both Claude and ChatGPT are active collaborators — cross-review welcome
- Upload this Master Project Summary at the start of every new conversation
- Patrick is not a professional programmer — explain at screenshot level
- Patrick is dyslexic, stroke survivor, blind in right eye, type 1 diabetic — keep messages clear, concise, and not visually overwhelming

Amendment (May 17, 2026): Surgical CTRL-F Edits for Large Files

For very large files like MainActivity.kt, full-file replacement carries real risk. A new approved workflow is permitted for small surgical changes:

- Claude provides an exact CTRL-F search string, long enough to be unique in the file
- Patrick searches and confirms exactly one match. If zero or two-plus matches, stop and report
- Claude provides the replacement text

This applies only to small surgical changes. New files and major rewrites still ship as full files.

Amendment (May 22, 2026): Multi-Step CTRL-F Instructions

For edits that span multiple lines or require deleting a block of code, Claude may give detailed multi-step instructions within a single edit block. Patrick confirms each full edit block before moving to the next.

Amendment (May 27, 2026): Always Click the Right File Tab

Android Studio CTRL-F only searches in the currently active editor tab. Every CTRL-F instruction must specify which file (scout_llama_api.h vs scout_llama_jni.cpp, etc.) and Patrick must click that file's tab BEFORE searching. Searching the wrong file wastes turns and causes confusion.

9. Key Files

File	Description
MainActivity.kt	Main app - all logic, inner classes, brain
ScoutFaceView.kt	Custom face canvas - all visual animation
GeminiClient.kt	Gemini HTTP wrapper with cooldown discipline
ScoutPromptBuilder.kt	Builds Gemini system instruction
ScoutGeminiManager.kt	Gemini orchestration
ScoutWeatherManager.kt	Live weather via Open-Meteo
ScoutPresenceDecider.kt	Social timing layer
ScoutIntentRouter.kt	Intent type routing
LlamaEngine.kt	Offline brain JNI wrapper - WORKING
OfflinePromptBuilder.kt	TinyLlama ChatML prompt formatter
scout_llama_jni.cpp	C++ JNI bridge - compiled into libscout_llama.so
scout_llama_api.h	Self-contained b8946 declarations
CMakeLists.txt	NDK build config
TeachExtractor.kt	Extracts facts like names from speech
HabitLayer.kt	Pattern memory - 14-day decay

10. Settings Architecture

The Settings screen is the user's control center for Scout. Five logical sections, built around the principle that defaults are calm and safe — users opt in to more capability rather than opt out.

10.1 Identity & Voice

- Robot Name — default "Scout", users can rename
- Voice pitch slider
- Voice speed slider
- Future voice tone options

10.2 Brain & Behavior

- Offline Mode (default ON) — Scout uses TinyLlama by default
- Online Brain Helper — toggle Gemini or a larger local model
- API key entry — user's own free Gemini key, never bundled
- Kid Safe Filter
- Pet Safety Protocol — includes Nicolas Protocol enforcement
- Presence Mode (default ON) — Scout actively listens in the room
- Allow Spontaneous Comments
- Privacy Mode toggle

10.3 Builder's Workbench

- Enable Hardware Mode (off by default)
- Bluetooth pairing — for KEYESTUDIO chassis
- Future motor controls

10.4 Privacy & Data

- Memory Export — back up TruthDb and habits
- Memory Import
- Reset Memory Layers — selective reset
- Camera controls
- Voice camera commands

10.5 Extras & Support

- Cosmetics (Backpack) — visual customization
- Support option
- About & Licenses

10.6 Connected Services (Future - All Opt-In)

- Calendar access — add, remove, and announce events. Uses Android Calendar Provider.
- Phone call awareness — Scout announces caller name then steps aside.
- Gmail access — read emails and compose when asked. Requires Google OAuth.

Design principle: Scout announces and helps, but never interferes.

11. Hardware Direction - Optional

Scout works fully without hardware. The Builder's Workbench in Settings is opt-in. Reference hardware: KEYESTUDIO Mini Tank Kit V2 (Patrick already owns one).

Relevant Specifications

- Tank-style chassis with motorized treads
- Ultrasonic obstacle avoidance sensors
- Light and ultrasonic follow capabilities
- 8×16 LED panel
- IR remote support
- Communication: Bluetooth

Why Hardware Is Opt-In

- A physically moving robot with weak presence feels emptier than a stationary companion with emotional realism. Hardware comes AFTER personality, emotional realism, local brain, stability, and presence are strong.
- Not every Scout owner wants hardware. The $9.99 baseline must work fully on the phone alone.

12. Future Vision - Scout Writes His Own Behavioral Code

A long-term goal that fits Scout's existing architecture naturally. Scout notices patterns in user behavior, generates a suggestion for a new behavior or routine, and asks permission before activating it. Nothing changes without the user's explicit consent.

Already Partially Designed In

- Proposal Sandbox layer was built exactly for this purpose
- Approval workflow already exists and is tested through the download system
- LLM (TinyLlama, upgraded model, or Gemini) generates the suggestion text
- TruthDb stores what gets approved permanently

Examples in Practice

- "You always ask about the weather at 7am. Want me to check it automatically each morning?"
- "You seem to talk to me less after 10pm. Want me to use softer responses at night?"
- "You ask about Elijah often on school days. Want me to remember his schedule?"

Important Technical Boundary

Scout cannot write and deploy actual compiled Kotlin code on device. Android does not allow that safely. What Scout CAN do is generate behavioral scripts, response patterns, routing rules, and habit triggers that change his behavior within the existing safe framework.

13. Scout Animation Goal

Scout's face should feel alive and emotionally present, but always calm. The goal is tiny layered movements that suggest thinking and presence — not cartoon performance.

Animation Principles

- Eyes slowly drift toward the speaker or point of interest.
- Pupils and irises use smooth easing — no instant jumps.
- Eyebrows slightly lift, lower, or tilt depending on mood.
- Blinks happen naturally, sometimes as a double-blink.
- Mouth makes small soft changes — never exaggerated expressions.
- Idle animation is very subtle: breathing-like motion, tiny eye shifts, rare blinks.

What Scout Should NOT Do

- Do NOT make Scout hyper, bouncy, or cartoonish.
- Scout should feel like a calm family companion watching, listening, and thinking.
- No Pixar-style exaggeration. No snappy or instant transitions.

Mood System - Foundation Code (Not Yet Wired In)

A mood-to-pose system has been designed and reviewed. It defines five emotional states. Each mood maps to a target FacePose with specific values for eye position, eyebrow lift, eyebrow tilt, mouth smile, and blink. Interpolation toward the target pose handles smooth transitions.

- **CALM** — Eyes center, brows neutral, subtle smile (0.15). Scout's default state.
- **CURIOUS** — Eyes shift slightly right and up, one brow lifts (0.18), slight tilt (0.12).
- **HAPPY** — Eyes center, both brows lift gently (0.12), smile increases (0.35).
- **THINKING** — Eyes shift left and down, brows slightly lower (-0.05), minimal smile (0.05).
- **CONCERNED** — Eyes look slightly down, inner brows lift (0.05), brows tilt inward (-0.18).

Status: Designed. Pending ScoutFaceView.kt integration.

14. Eye & Mouth Movement Stability & Realism Notes

Scout's eyes and mouth are intentionally designed to have tiny natural motion. Completely frozen facial features feel lifeless, while exaggerated movement feels cartoonish. The goal is "quietly alive."

Current Visual Issues Observed

- During startup, thinking states, or heavy animation activity, Scout's irises may jitter or wiggle rapidly.
- Scout's mouth currently behaves more like a looping animation — opening and closing repeatedly.
- These behaviors appear to be caused by overlapping animation targets, frame timing spikes, or rapid target updates.
- CONFIRMED May 29, 2026: dropping n_threads to 2 did NOT fix the jitter. Generation already runs on a background thread, so this is an animation-code issue in ScoutFaceView, not a CPU/thread issue. Heavy inference only makes it more visible. Needs a dedicated ScoutFaceView session.

Desired Direction

- Eye movement should remain calm, slow, and emotionally subtle.
- Iris movement should use smoothing/easing rather than sudden jumps.
- Tiny micro-movements are encouraged for realism.
- Mouth movement should react naturally to speech volume/amplitude rather than looping open/closed.
- When speaking ends, the mouth should gently settle back into a neutral resting pose.

Important Boundary

Scout should NEVER have perfectly still eyes or a perfectly static mouth. Stillness feels emotionally dead. The target feeling is subtle organic presence — calm, stable, observant, thoughtful, and quietly alive.

15. Device Requirements & Play Store Expectations

Decided May 29, 2026 (cross-reviewed by Claude and ChatGPT). The Samsung Galaxy A32 is Scout's stress-test device — not the benchmark. If Scout runs stably on a 2021 budget phone juggling Android, face animation, STT, TTS, camera, memory, and a language model simultaneously, that is a sign of resilience. Newer phones will feel like a meaningfully better product.

Minimum Requirements

Devices where Scout should function correctly:

- Android 13 or newer
- 4 GB RAM minimum (6 GB recommended for comfort — 4 GB may be tight with background apps)
- Approximately 3–5 GB free storage (depending on downloaded brain packs)
- Internet connection required for optional cloud AI features
- Offline mode supported with downloaded brain packs

Expected experience at minimum spec: Face animations work. Voice works. Memory works. Offline AI works. Responses may be noticeably slower on older devices.

Recommended Requirements

Devices that give the experience Scout is designed for:

- Android 13 or newer
- 8 GB RAM or more
- Modern mid-range or flagship processor (2023 or newer)
- 10 GB+ free storage for future brain packs
- Stable Wi-Fi connection for cloud AI features

Expected experience at recommended spec: Faster responses. Smoother animations. Better multitasking. Improved offline AI performance.

Play Store Listing Wording (Agreed)

"Offline AI performance varies by device. Scout runs on many Android devices, but newer phones and tablets provide faster response times and smoother animations."

This sets expectations honestly without scaring people away. It protects against bad reviews from users on very old hardware.

What the Public Will Actually Judge Scout By

The public will not judge Scout by tokens per second. They will judge Scout by:

- Does he remember my name?
- Does he greet me?
- Does he feel alive?
- Does he look at me?
- Does he respond consistently?
- Does he feel like a companion?

Scout's value is presence, memory, and calm behavior — not raw AI throughput. A 20-second offline response that says 'I haven't seen you today. How has your day been?' will be tolerated far more than a 20-second generic factual answer. The architecture is correct. The offline vision is achievable.

This document is the single source of truth for Project Scout. Begin every new Claude or ChatGPT conversation by uploading this file.